Initial sequential scan experiment

Radix hash join project | Windows 11 measurements

We wanted to see how the gap between memory reads changes the time a scan takes. Every scan read 200,000 bytes, one at a time. After clearing the selected data from cache, the scan took about 11 times as long at stride 256 as it did at stride 1.

How we ran the test

We filled a 256 MiB array with random-looking byte values that we could reproduce. The stride is the gap between reads. For example, at stride 8 the program reads bytes 0, 8, 16 and so on. We tested 11 strides from 1 to 1024 bytes, doubling the gap each time.

Flushed scan. Before starting the timer, we removed the data the scan would read from the CPU caches. As the scan ran, the CPU could bring that data back into cache.

Immediate repeat scan. We read the same positions once, then timed a second scan straight away. Some of the data from the first scan could still be in cache.

We ran three warmup rounds and then recorded 30 runs for each stride and method. We changed the order each round and ran the test on one logical processor. The timer covered reading and adding the bytes. We checked the sum after each scan, and all 660 recorded scans passed.

We followed the scan setup in Section 2 of the paper [1]. The laptop used an AMD Ryzen 7 7840HS with 64-byte cache lines, 32 KiB of L1 data cache and 1 MiB of L2 per core, and 16 MiB of shared L3 cache.

How the stride affected scan time

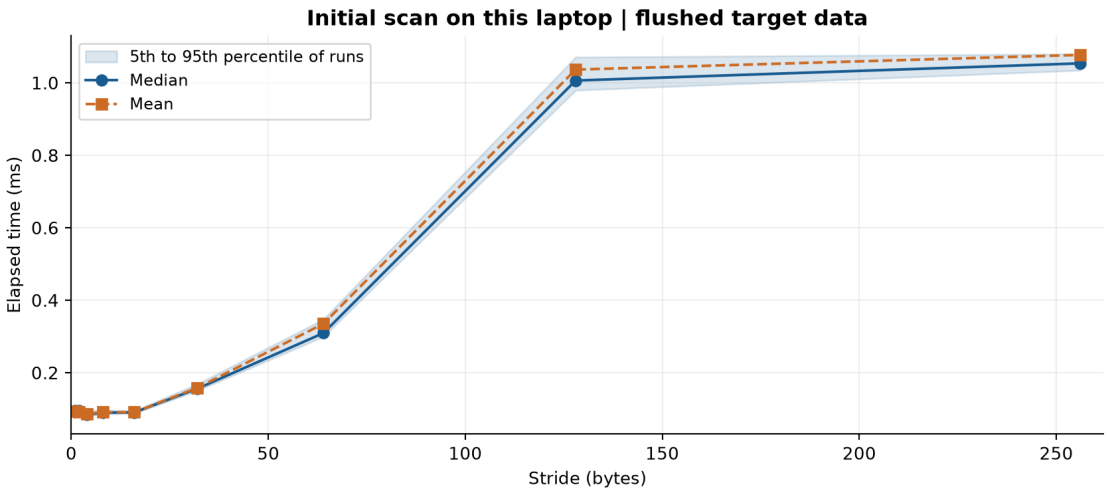


Figure 1. Time for the whole scan after clearing the selected data from cache. The blue line shows the median, the orange line shows the average, and the shaded band covers the middle 90% of measured times.

What we saw. The median stayed near 0.09 ms for strides 1 to 16. The biggest jump was from 0.309 ms at stride 64 to 1.006 ms at stride 128. At stride 256 it reached 1.053 ms. You can see this in the blue line, which climbs sharply between 64 and 128 and then starts to level off. The shaded band is wider at stride 64 than at stride 16, so the timings varied more there.

How to read it. When reads are close together, several of them can use the same 64-byte cache line. At stride 64 and above, each read uses a different line. Strides 64 and 128 touch the same number of lines, so the jump between them suggests that the spacing between lines also matters. The scan does the same number of reads at every stride, so the extra time comes from changing how it visits memory. The chart gives us a reason to pay attention to where the join keeps the data it looks up.

Scan times by stride

Stride B	Mean ms	Median ms	SD ms	P95 ms	Min ms	Max ms
1	0.0929	0.0955	0.0075	0.1024	0.0804	0.1082
2	0.0911	0.0952	0.0066	0.0958	0.0804	0.1014
4	0.0862	0.0837	0.0054	0.0955	0.0806	0.0960
8	0.0906	0.0893	0.0069	0.0963	0.0828	0.1216
16	0.0906	0.0902	0.0031	0.0945	0.0863	0.1015
32	0.1581	0.1557	0.0066	0.1684	0.1481	0.1708
64	0.3349	0.3090	0.1077	0.3469	0.2983	0.8999
128	1.0362	1.0060	0.1522	1.0705	0.9769	1.8323
256	1.0767	1.0534	0.1237	1.0795	1.0263	1.7276

SD describes how much the times varied. P95 is the 95th percentile.

Did reading the data beforehand help?

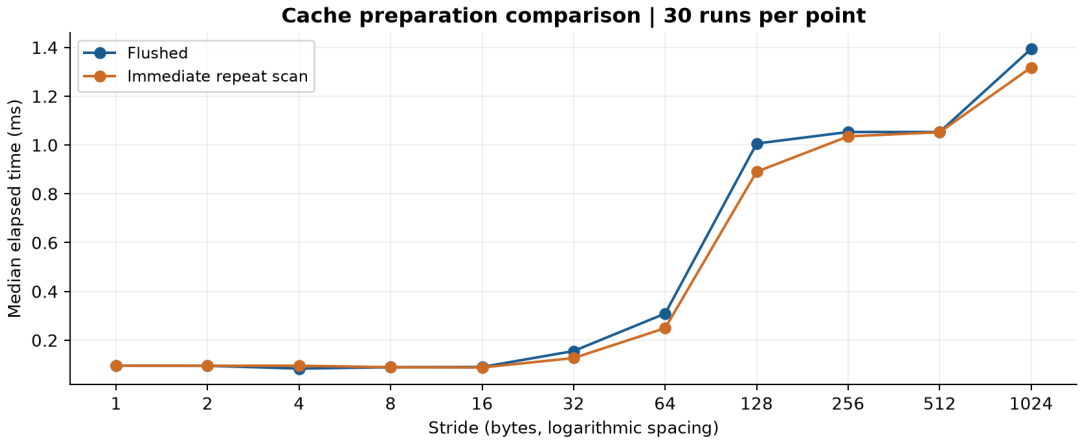


Figure 2. Median scan times for both methods. Each step along the horizontal axis doubles the stride.

What we saw. At stride 64, reading the data beforehand brought the median down from 0.3090 ms to 0.2495 ms. At stride 512 the two times were nearly the same. Both methods slowed down at stride 1024, where the repeat scan was slightly quicker again. The gap between the blue and orange lines is easiest to see at stride 128. From 256 to 512, the lines almost overlap.

How to read it. Reading the data first helped at some strides. The gap between reads still had a bigger effect on the overall time. At stride 1024, the reads span about 195 MiB of addresses, but they skip most of that space. They touch about 12.2 MiB of cache lines, the same amount as at stride 256. The timings show a slowdown as the reads spread farther apart. They do not tell us exactly what caused it.

What this means for our radix hash join

A simple hash join can jump around one large hash table as it looks for matches. Radix partitioning groups matching keys so we can join one pair of partitions at a time. The smaller hash tables give recently used data a better chance of staying in cache. Figure 2 suggests that reuse can help, although the benefit depends on how we access the data.

We can test this by running the simple join and the radix join on the same inputs at several sizes. We should measure partitioning, building the hash tables, and probing for matches separately. Then we can compare total times to see where faster lookups save enough time to pay for partitioning.

[1] P. Boncz, S. Manegold and M. L. Kersten. Database Architecture Optimized for the New Bottleneck: Memory Access. Proceedings of VLDB, 1999. Sections 2 and 3.